



Flow-based Deep Generative Models (Part 3)

<https://hku-data8018.github.io/>

Bo Dai

Spring 2026



- Slides reference
 - An introduction to flow matching, <https://mlg.eng.cam.ac.uk/blog/2024/01/20/flow-matching.html>
 - <https://rectifiedflow.github.io/>
 - A Visual Introduction to Rectified Flows, <https://alechelbling.com/blog/rectified-flow/>

Flow matching: a **simulation-free** way to train continuous normalizing flows

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0,1]} \mathbb{E}_{x \sim p_t} [\|u_\theta(t, x) - u(t, x)\|^2].$$

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0,1], x_1 \sim q, x_t \sim p_t(x|x_1)} [\|u_\theta(t, x) - u_t(x | x_1)\|^2].$$

$$\nabla_\theta \mathcal{L}_{\text{FM}}(\theta) = \nabla_\theta \mathcal{L}_{\text{CFM}}(\theta),$$

Practical Example:

- A conditional vector field that generates a path of Gaussians with some mean function and deviation function

$$p_t(x \mid x_1) = \mathcal{N}(x; \mu_t(x_1), \sigma_t(x_1)^2 \mathbf{I})$$

$$u_t(x \mid x_1) = \frac{\dot{\sigma}_t(x_1)}{\sigma_t(x_1)}(x - \mu_t(x_1)) + \dot{\mu}_t(x_1)$$

- One choice of mean and deviation:

$$\begin{aligned} \mu_t(x_1) &\triangleq tx_1 & \text{and} & \quad \sigma_t(x_1) \triangleq (1-t) + t\sigma_{\min} \\ \dot{\mu}_t(x_1) &\triangleq x_1 & \text{and} & \quad \dot{\sigma}_t(x_1) \triangleq -1 + \sigma_{\min} \end{aligned}$$

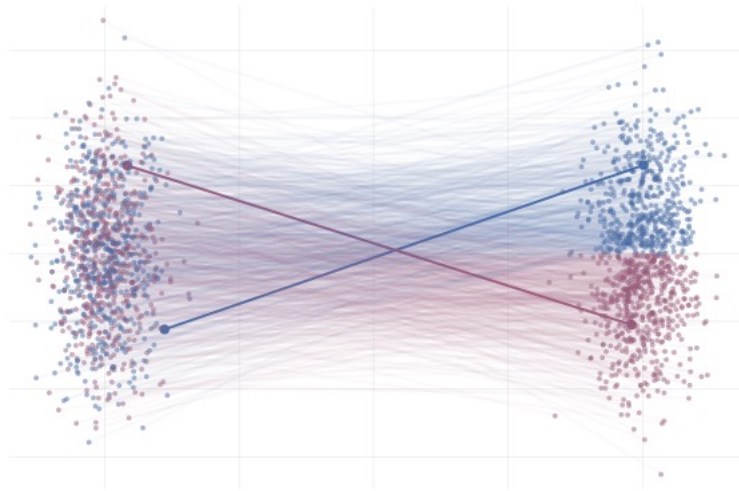
$$(\mu_0(x_1) + \sigma_0(x_1)x_1) \sim p_0 \quad \text{and} \quad (\mu_1(x_1) + \sigma_1(x_1)x_1) \sim \mathcal{N}(x_1, \sigma_{\min}^2 I)$$

$$u_t(x | x_1) = \frac{\dot{\sigma}_t(x_1)}{\sigma_t(x_1)} (x - \mu_t(x_1)) + \dot{\mu}_t(x_1)$$

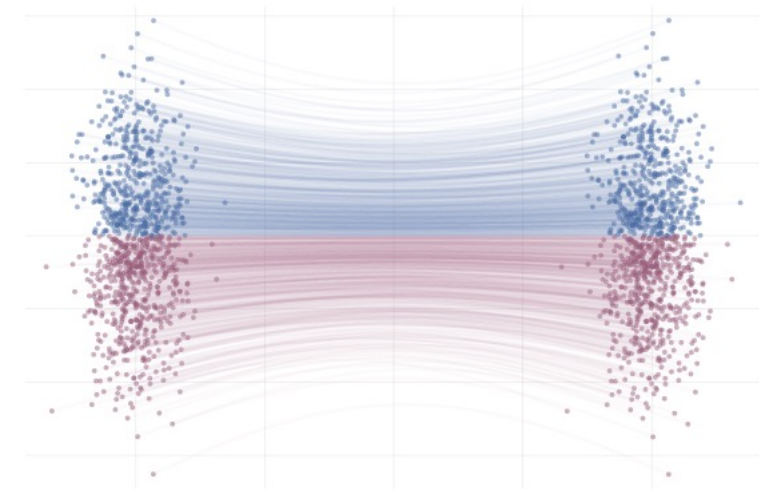
Practical Example:

$$\begin{aligned} \mu_t(x_1) &\triangleq tx_1 & \text{and} & \sigma_t(x_1) \triangleq (1-t) + t\sigma_{\min} \\ \dot{\mu}_t(x_1) &\triangleq x_1 & \text{and} & \dot{\sigma}_t(x_1) \triangleq -1 + \sigma_{\min} \end{aligned}$$

- Visualization of the difference between the marginal and conditional flows.



Realizations of paths from p_0 to p_1 following **conditional vector fields** $u_t(x|x_1)$. Paths are highlighted by the sign of the 2nd vector component at time $t = 1$.



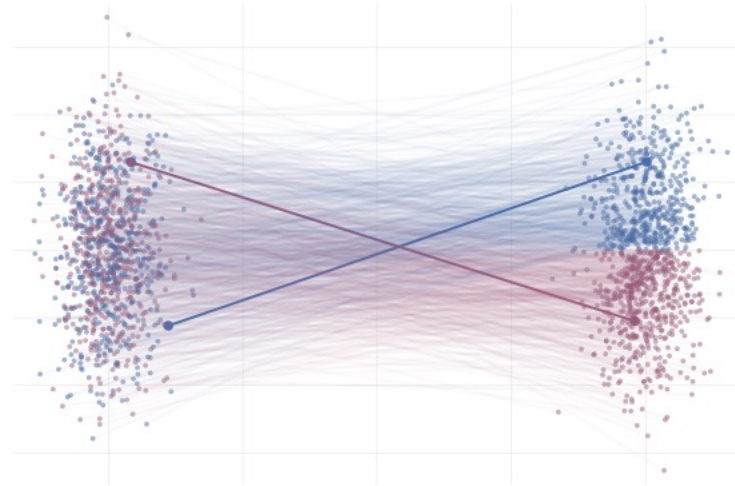
Paths from p_0 to p_1 following **the true marginal vector field** $u_t(x)$. Paths are highlighted by the sign of the 2nd vector component.

Issues of Conditional Flow Matching: **crossing conditional paths.**



- Many possible x_1 for a noised $x_t \rightarrow$ high CFM loss variance $\rightarrow$ slow training convergence

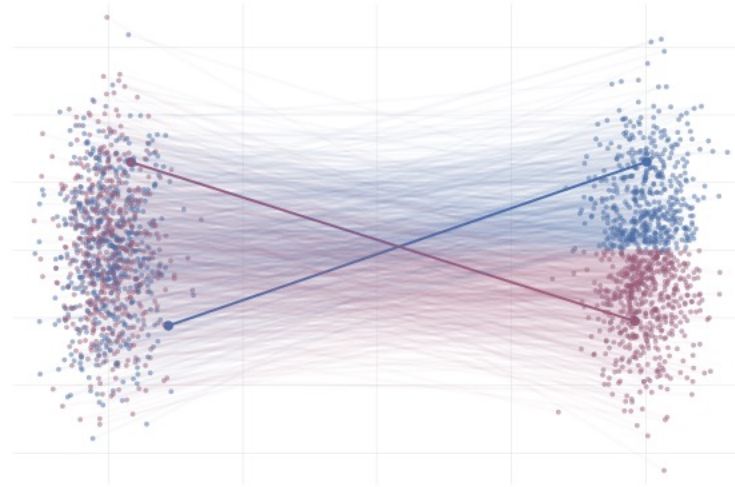
Issues of Conditional Flow Matching: **crossing conditional paths.**



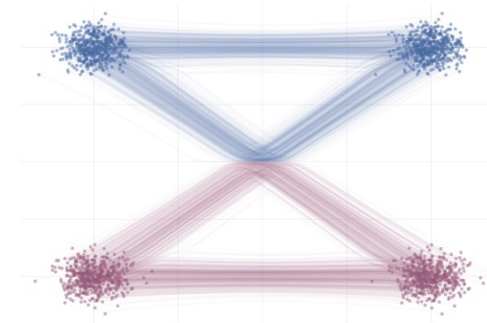
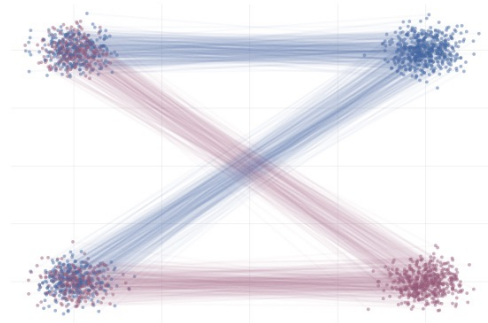
- Many possible x_1 for a noised $x_t \rightarrow$ high CFM loss variance $\rightarrow$ slow training convergence
 - Paths following the same x_1 will not cross, but paths following different x_1 will

$$\mathcal{L}_{\text{CFM}}(\theta) \approx \frac{1}{2} \left\| u_{\theta}(t, x_t^{(1)}) - u(t, x_t^{(1)} \mid x_1^{(1)}) \right\| + \frac{1}{2} \left\| u_{\theta}(t, x_t^{(2)}) - u(t, x_t^{(2)} \mid x_1^{(2)}) \right\|$$

Issues of Conditional Flow Matching: **crossing conditional paths.**



- Many possible x_1 for a noised $x_t \rightarrow$ high CFM loss variance $\rightarrow$ slow training convergence
- **Non-straight marginal paths $\rightarrow$ ODE hard to integrate $\rightarrow$ slow sampling at inference.**
 - At training time paths may intersect, whilst at sampling time they cannot



Issues of Conditional Flow Matching: **crossing conditional paths**.

- One-sided conditioning:

$$p_t(x \mid x_1) = \mathcal{N}(x; \mu_t(x_1), \sigma_t(x_1)^2 \mathbf{I})$$

- **Coupling: two-sided conditioning**

$$p_t(x_t) = \int p_t(x_t \mid z) q(z) \, dz = \int p_t(x_t \mid x_1, x_0) q(x_1, x_0) \, dx_1 \, dx_0.$$

The following boundary condition on $p_t(x_t \mid x_1, x_0)$: $p_0(\cdot \mid x_1, x_0) = \delta_{x_0}$ and $p_1(\cdot \mid x_1, x_0) = \delta_{x_1}$ is required so that the marginal has the proper conditions $p_0 = q_0$ and $p_1 = q_1$.

Issues of Conditional Flow Matching: **crossing conditional paths.**

- One-sided conditioning:

$$p_t(x \mid x_1) = \mathcal{N}(x; \mu_t(x_1), \sigma_t(x_1)^2 \mathbf{I})$$

- Coupling: two-sided conditioning

$$p_t(x_t) = \int p_t(x_t \mid z) q(z) \, dz = \int p_t(x_t \mid x_1, x_0) q(x_1, x_0) \, dx_1 \, dx_0.$$

- **One example: optimal transport coupling. Mini-batch OT in practice**

$$q(x_1, x_0) = \pi(x_1, x_0) \in \arg \inf_{\pi \in \Pi} \int \|x_1 - x_0\|_2^2 d\pi(x_1, x_0)$$

Issues of Conditional Flow Matching: **crossing conditional paths**.

- One-sided conditioning:

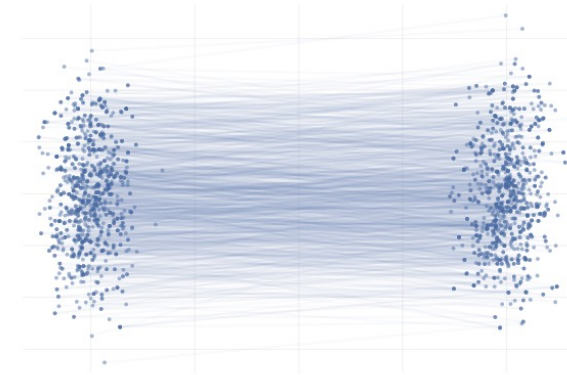
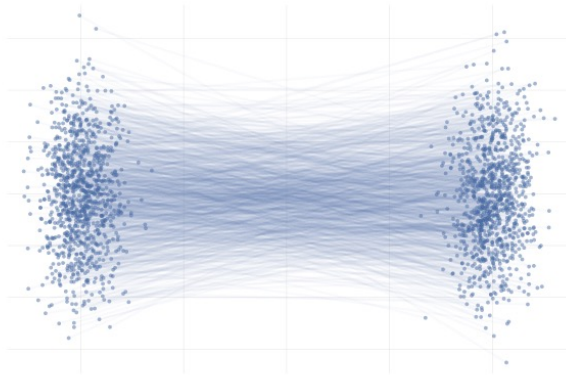
$$p_t(x \mid x_1) = \mathcal{N}(x; \mu_t(x_1), \sigma_t(x_1)^2 \mathbf{I})$$

- Coupling: two-sided conditioning

$$p_t(x_t) = \int p_t(x_t \mid z) q(z) \, dz = \int p_t(x_t \mid x_1, x_0) q(x_1, x_0) \, dx_1 \, dx_0.$$

- One example: optimal transport coupling. Mini-batch OT in practice

$$q(x_1, x_0) = \pi(x_1, x_0) \in \arg \inf_{\pi \in \Pi} \int \|x_1 - x_0\|_2^2 d\pi(x_1, x_0)$$



Issues of Conditional Flow Matching: **crossing conditional paths**.

- Many possible x_1 for a noised $x_t \rightarrow$ high CFM loss variance $\rightarrow$ slow training convergence
- Non-straight marginal paths $\rightarrow$ ODE hard to integrate $\rightarrow$ slow sampling at inference.

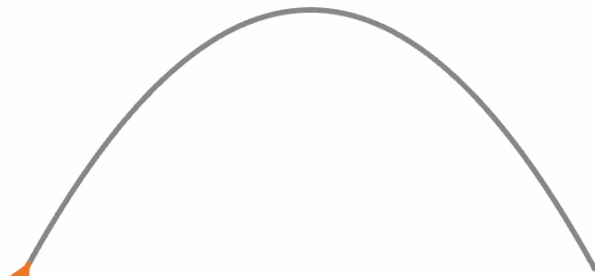
Rectified Flow:

- Many valid probability flows connecting p_0 and p_1 , the one **with straight-line trajectories** lead to zero **discretization error**

$$\dot{Z}_t = v_t(Z_t), \quad \forall t \in [0, 1], \quad \text{starting from } Z_0 \sim \pi_0,$$

$$Z_t = tZ_1 + (1 - t)Z_0, \quad \implies \quad \dot{Z}_t = Z_1 - Z_0.$$

II Curved Function



Almost Straight Function



Rectified Flow:

- Many valid probability flows connecting p_0 and p_1 , the one **with straight-line trajectories** lead to zero **discretization error**

- **Build Interpolation:**

The first step is to build an interpolation process $\{X_t\} = \{X_t : t \in [0, 1]\}$ that smoothly interpolates between X_0 and X_1 . Although general choices are possible, let us consider the canonical choice of straight-line interpolation:

$$X_t = tX_1 + (1 - t)X_0.$$

Here the interpolation $\{X_t\}$ is a stochastic process generated in an “**anchor-and-bridge**” way: we first sample the endpoints X_0 and X_1 and then sample the intermediate trajectory connecting them.

Rectified Flow:

- Many valid probability flows connecting p_0 and p_1 , the one **with straight-line trajectories** lead to zero **discretization error**
 - Build Interpolation

$$X_t = tX_1 + (1 - t)X_0.$$

- **Marginal Matching:**

By construction, the marginal distributions of X_0 and X_1 match the target distributions π_0 and π_1 through the interpolation process $\{X_t\}$. However, $\{X_t\}$ is not a *causal* ODE process like $\dot{Z}_t = v_t(Z_t)$, which generate the output Z_1 by evolving forward in time from Z_0 . Instead, generating X_t requires knowledge of both X_0 and X_1 , rather than evolving solely from X_0 as t increases.

Rectified Flow:

- Many valid probability flows connecting p_0 and p_1 , the one **with straight-line trajectories** lead to zero **discretization error**
 - Build Interpolation

$$X_t = tX_1 + (1 - t)X_0.$$

- Marginal Matching:**

By construction, the marginal distributions of X_0 and X_1 match the target distributions π_0 and π_1 through the interpolation process $\{X_t\}$. However, $\{X_t\}$ is not a *causal* ODE process like $\dot{Z}_t = v_t(Z_t)$, which generate the output Z_1 by evolving forward in time from Z_0 . Instead, generating X_t requires knowledge of both X_0 and X_1 , rather than evolving solely from X_0 as t increases.

This issue can be resolved if we can convert $\{X_t\}$ somehow into a causal ODE process while preserving the marginal distributions of X_t at each time t . Note that since we only care about the output X_1 , we only need to match the marginal distributions of X_t at each individual time t . There is no need to match the trajectory-wise joint distribution of $\{X_t\}$.

Rectified Flow:

- Many valid probability flows connecting p_0 and p_1 , the one **with straight-line trajectories** lead to zero **discretization error**
 - Build Interpolation

$$X_t = tX_1 + (1 - t)X_0.$$

- Marginal Matching**

$$v_t^*(x) = \mathbb{E} \left[\dot{X}_t \mid X_t = x \right],$$

which is the conditional expectation of the slope $\dot{X}_t$ for all the interpolation trajectories passing through a given point $X_t = x$. If multiple trajectories pass point $X_t = x$, the velocity $v_t^*(x)$ is the average of $\dot{X}_t$ for these trajectories.

Rectified Flow:

- Many valid probability flows connecting p_0 and p_1 , the one **with straight-line trajectories** lead to zero **discretization error**
 - Build Interpolation

$$X_t = tX_1 + (1 - t)X_0.$$

- Marginal Matching

$$v_t^*(x) = \mathbb{E} \left[\dot{X}_t \mid X_t = x \right],$$

- Learning Objective over the slope:**

$$\min_v \int_0^1 \mathbb{E} \left[\|\dot{X}_t - v_t(X_t)\|^2 \right] dt, \quad X_t = tX_1 + (1 - t)X_0.$$

Rectified Flow:

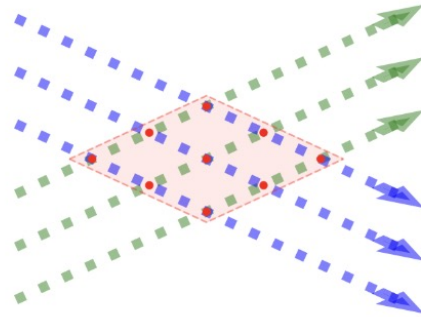
Rectified Flow. For any time-differential stochastic process $\{X_t\} = \{X_t : t \in [0, 1]\}$, we call the ODE process:

$$\dot{Z}_t = v_t^*(Z_t) \quad \text{with} \quad v_t^*(x) = \mathbb{E} \left[\dot{X}_t \mid X_t = x \right], \quad Z_0 = X_0$$

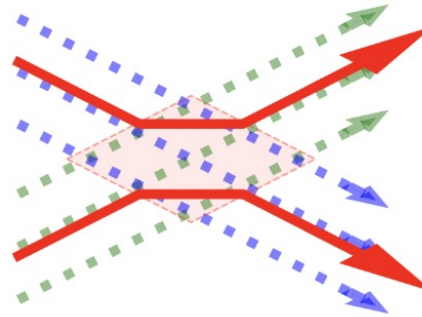
the **rectified flow** induced by $\{X_t\}$. We denote it as:

$$\{Z_t\} = \text{Rectify}(\{X_t\}).$$

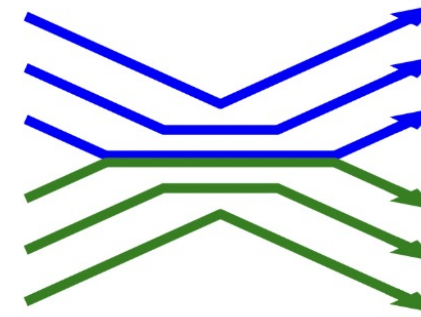
Rectified Flow:



(a)



(b)



(c)

Rectified Flow. For any time-differential stochastic process $\{X_t\} = \{X_t : t \in [0, 1]\}$, we call the ODE process:

$$\dot{Z}_t = v_t^*(Z_t) \quad \text{with} \quad v_t^*(x) = \mathbb{E}[\dot{X}_t \mid X_t = x], \quad Z_0 = X_0$$

the **rectified flow** induced by $\{X_t\}$. We denote it as:

$$\{Z_t\} = \text{Rectify}(\{X_t\}).$$

- In the interpolation process $\{X_t\}$, different trajectories may have intersecting points, resulting in multiple possible values of $\dot{X}_t$ associated with a same point X_t due to uncertainty about which trajectory it was drawn from
- In contrast, by the definition of an ODE $\dot{Z}_t = v_t^*(Z_t)$, the update direction $\dot{Z}_t$ at each point Z_t is uniquely determined by Z_t , making it impossible for different trajectories of Z_t to intersect and then diverge along different directions.
- Hence at these intersection points of $\{X_t\}$ where $\{\dot{X}_t\}$ is uncertain and non-unique, the ODE $\{Z_t\}$ “derandomizes” the update direction by following the conditional expectation $v_t^*(X_t) = \mathbb{E}[\dot{X}_t \mid X_t]$. Consequently, the trajectories of the ODE “reassemble” the interpolation trajectories in a way that avoids intersections
- Since ODE trajectories Z_t cannot intersect, they must curve at potential intersection points to “rewire” the original interpolation paths and avoid crossing.

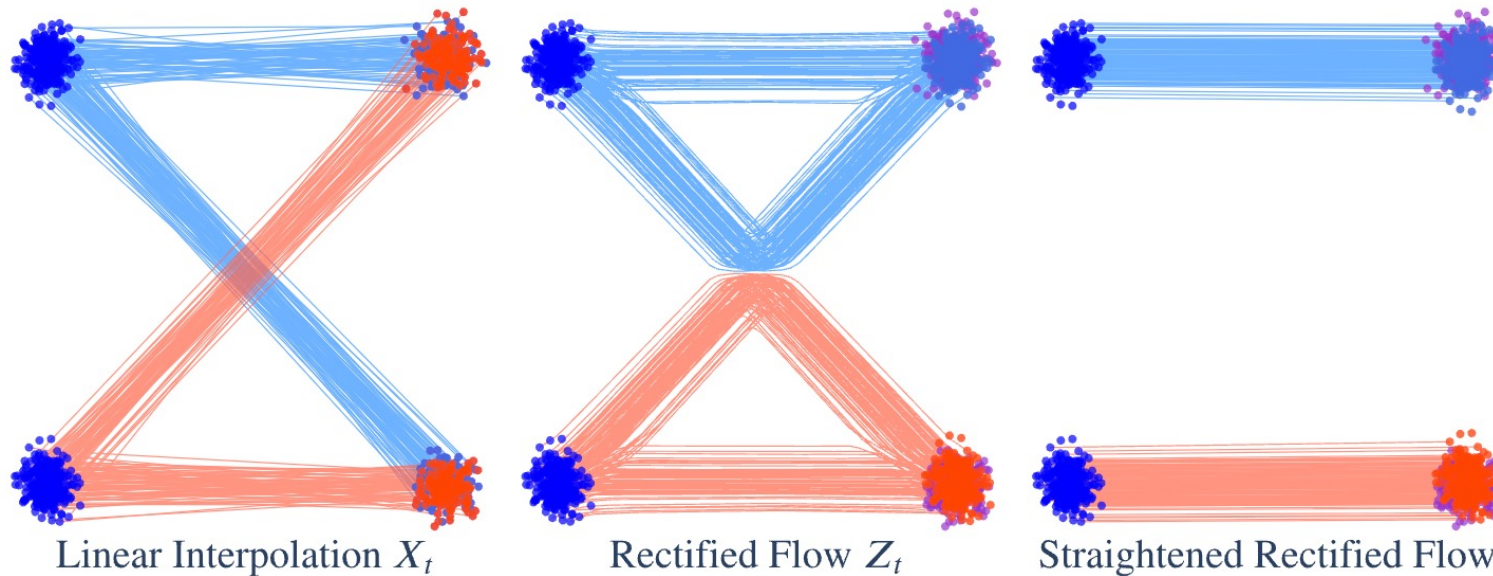
Reflow:

- rectified flows tend to favor straight trajectories, they are **not perfectly straight**.
- the start-end pairs (Z_0, Z_1) generated by rectified flow, called the **rectified coupling** of (X_0, X_1) , form a **better and “straighter” coupling** compared to (X_0, X_1) .

Reflow:

- rectified flows tend to favor straight trajectories, they are not perfectly straight.
- the start-end pairs (Z_0, Z_1) generated by rectified flow, called the **rectified coupling** of (X_0, X_1) , form a better and “straighter” coupling compared to (X_0, X_1) .
- **Apply it recursively!**

$$\text{Reflow: } \{Z_t^{k+1}\} = \text{Rectify}(\text{Interp}(Z_0^k, Z_1^k)),$$



Flow-based models v.s. Diffusion models

- Flow-based models provides a **deterministic ODE** (a.k.a. flow) model
- Diffusion models rely on a **stochastic differential equation (SDE)** to generate data from noise
- Questions:
 - *Why and how is it possible to convert between SDEs and ODEs? What is the intuition?*
 - *Why would we bother to add diffusion noise given that ODEs are simpler and faster? What are the pros and cons of diffusion vs. flow?*

Flow-based models v.s. Diffusion models

- Rectified flow

$$\dot{Z}_t = v_t^*(Z_t) \quad \text{with} \quad v_t^*(x) = \mathbb{E} \left[\dot{X}_t \mid X_t = x \right], \quad Z_0 = X_0$$

- **Ideally**, scheduled to do the right thing at the right time, the process guarantees the correct final result.

Flow-based models v.s. Diffusion models

- Rectified flow

$$\dot{Z}_t = v_t^*(Z_t) \quad \text{with} \quad v_t^*(x) = \mathbb{E} \left[\dot{X}_t \mid X_t = x \right], \quad Z_0 = X_0$$

- **Ideally**, scheduled to do the right thing at the right time, the process guarantees the correct final result.
- **In practice**, drift happens between the estimated distribution and the true distribution.
 - numerical discretization
 - model approximation
 - ...
- **Compound effect**: if the estimated trajectory deviates significantly from the distribution of X_t , the update direction becomes less accurate and can further reinforce the deviation.

Flow-based models v.s. Diffusion models

- Rectified flow

$$\dot{Z}_t = v_t^*(Z_t) \quad \text{with} \quad v_t^*(x) = \mathbb{E} [\dot{X}_t \mid X_t = x], \quad Z_0 = X_0$$

- In practice**, drift happens between the estimated distribution and the true distribution.
- Introducing a **feedback mechanism** to correct drifts. One such approach is to use **Langevin dynamics**.

Langevin Dynamics. For a density function $\rho^*(x)$, its (discrete-time) Langevin dynamics is

$$\hat{Z}_{t+\epsilon} = \hat{Z}_t + \epsilon \sigma_t^2 \nabla \log \rho^*(\hat{Z}_t) + \sqrt{2\epsilon} \sigma_t \xi_t, \quad \xi_t \sim \text{Normal}(0, I),$$

where σ_t is the diffusion coefficient, and $\epsilon > 0$ is the step size. Intuitively, this update is gradient ascent on log probability $\log \rho^*$ with Gaussian noise perturbations of variance ϵ at each step. Langevin dynamics provides an approximate method to draw samples from ρ^* because, under regularity conditions, the distribution of Z_t converges to ρ^* as $t \rightarrow +\infty$ and $\epsilon \rightarrow 0$.

Flow-based models v.s. Diffusion models

- Rectified flow

$$\dot{Z}_t = v_t^*(Z_t) \quad \text{with} \quad v_t^*(x) = \mathbb{E} \left[\dot{X}_t \mid X_t = x \right], \quad Z_0 = X_0$$

- **In practice**, drift happens between the estimated distribution and the true distribution.
- Introducing a **feedback mechanism** to correct drifts. One such approach is to use **Langevin dynamics**.
 - Applying a short segment of Langevin dynamics to adjust. A double loop.

$$dZ_{t,\tau} = \sigma_t^2 \nabla \log \rho_t(Z_{t,\tau}) d\tau + \sqrt{2} \sigma_t dW_\tau, \quad \tau \geq 0,$$

Flow-based models v.s. Diffusion models

- Rectified flow

$$\dot{Z}_t = v_t^*(Z_t) \quad \text{with} \quad v_t^*(x) = \mathbb{E} \left[\dot{X}_t \mid X_t = x \right], \quad Z_0 = X_0$$

- In practice, drift happens between the estimated distribution and the true distribution.
- Introducing a **feedback mechanism** to correct drifts. One such approach is to use **Langevin dynamics**.
 - Applying a short segment of Langevin dynamics to adjust. A double loop.
 - **Since rectified flow has only small drift**, a single step of Langevin dynamics is sufficient.

$$dZ_t = \underbrace{v_t(Z_t) dt}_{\text{Rectified Flow}} + \underbrace{\sigma_t^2 \nabla \log \rho_t(Z_t) dt + \sqrt{2} \sigma_t dW_t}_{\text{Langevin Dynamics}}, \quad \tilde{Z}_0 = Z_0.$$

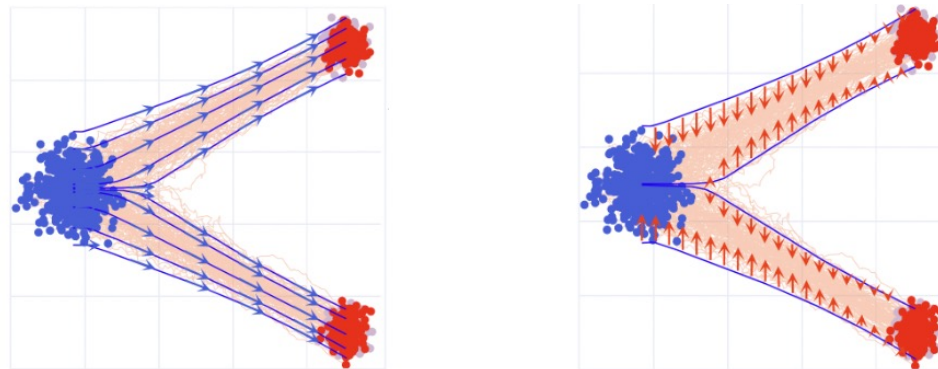
Flow-based models v.s. Diffusion models

- Rectified flow

$$\dot{Z}_t = v_t^*(Z_t) \quad \text{with} \quad v_t^*(x) = \mathbb{E} [\dot{X}_t \mid X_t = x], \quad Z_0 = X_0$$

- Introducing a **feedback mechanism** to correct drifts. One such approach is to use **Langevin dynamics**.
 - Since rectified flow has only small drift, a single step of Langevin dynamics is sufficient.

$$dZ_t = \underbrace{v_t(Z_t) dt}_{\text{Rectified Flow}} + \underbrace{\sigma_t^2 \nabla \log \rho_t(Z_t) dt + \sqrt{2} \sigma_t dW_t}_{\text{Langevin Dynamics}}, \quad \tilde{Z}_0 = Z_0.$$



- The **rectified flow** drives the generative process forward as intended.
- The **Langevin component** acts as a negative feedback loop, correcting distributional drift without bias when $\tilde{Z}_t$ and ρ_t are well aligned.

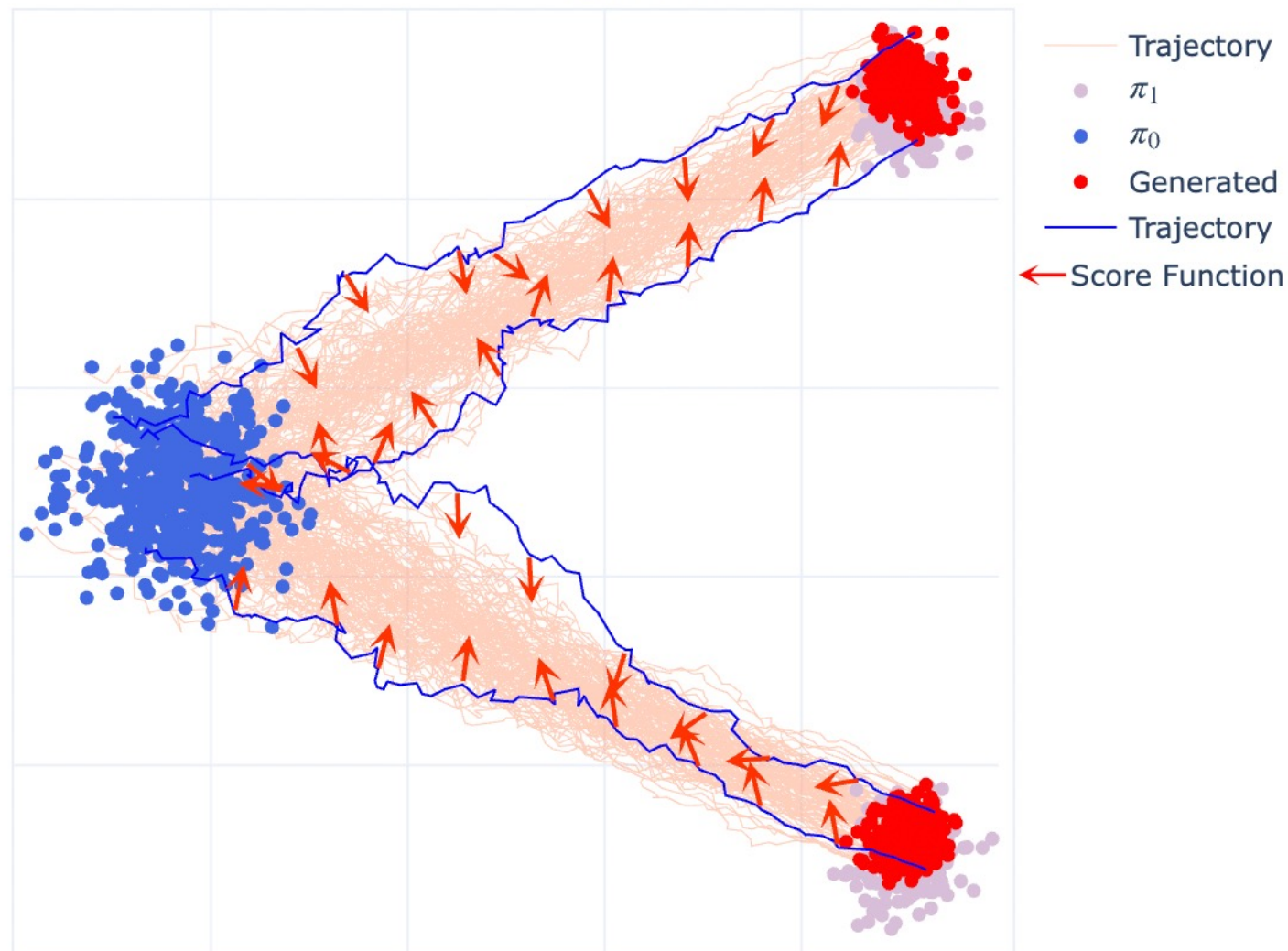
Score Function $\nabla \log \rho_t(x)$ in SDE Trajectories

Flow-based m

- Rectified fl

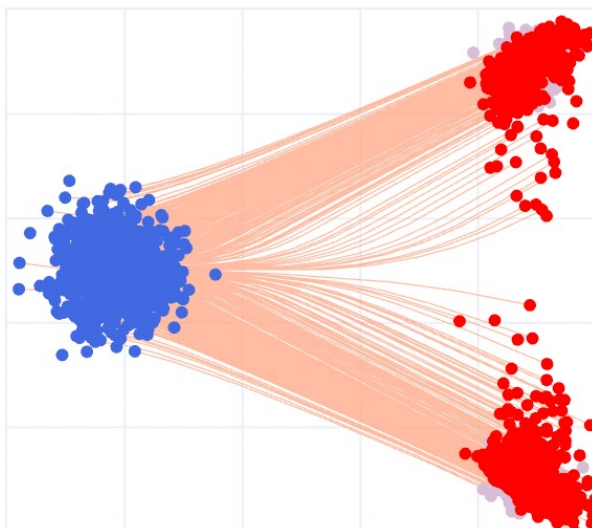
- Intro

- :

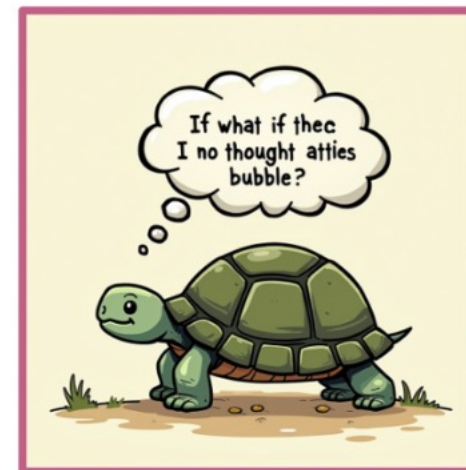
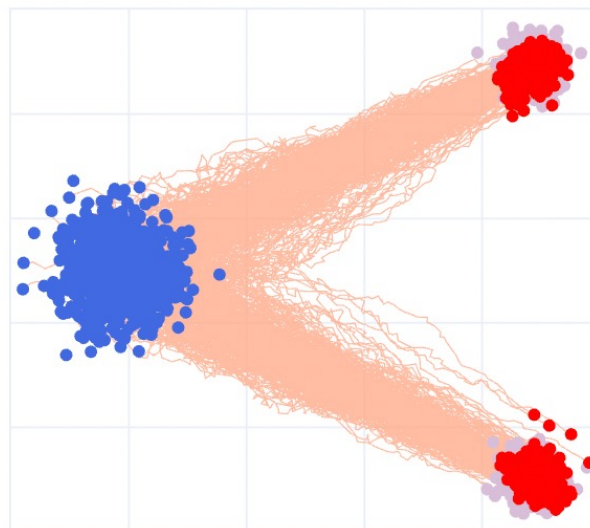


amics.

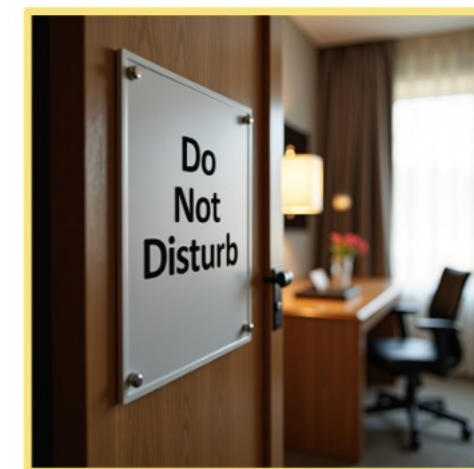
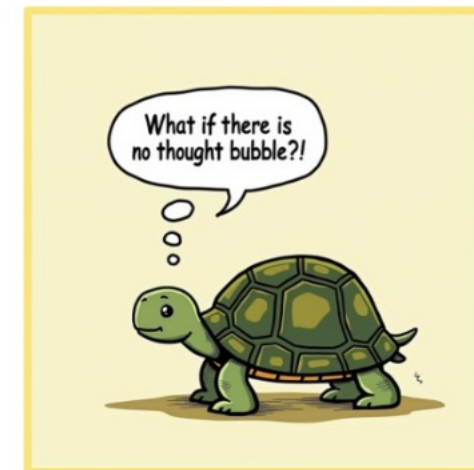
Deterministic Sampler



Stochastic Sampler



Prompt: A cartoon of a turtle with a thought bubble on its head and the words "**what if there is no thought bubble**"



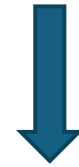
Prompt: There is a "**Do Not Disturb**" sign in the hotel

Flow-based models v.s. Diffusion models

$$dZ_t = \underbrace{v_t(Z_t) dt}_{\text{Rectified Flow}} + \underbrace{\sigma_t^2 \nabla \log \rho_t(Z_t) dt + \sqrt{2} \sigma_t dW_t}_{\text{Langevin Dynamics}}, \quad \tilde{Z}_0 = Z_0.$$

- How to compute $\nabla \log \rho_t$ from $v_t(Z_t)$? Using the **Tweedie's formula**

$$\nabla \log \rho_t(x) = -\frac{1}{\beta_t} \mathbb{E}[X_0 \mid X_t = x].$$



$$v_t(x) = \mathbb{E}[\dot{X}_t \mid X_t = x] = \mathbb{E}[\dot{\alpha}_t X_1 + \dot{\beta}_t X_0 \mid X_t = x].$$

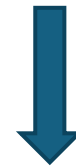
$$X_t = \alpha_t X_1 + \beta_t X_0,$$

$$\nabla \log \rho_t(x) = \frac{\alpha_t v_t(x) - \dot{\alpha}_t x}{\lambda_t \beta_t}, \quad \text{where } \lambda_t = \dot{\alpha}_t \beta_t - \alpha_t \dot{\beta}_t.$$

Flow-based models v.s. Diffusion models

- A training-free conversion between ODEs and SDEs

$$dZ_t = \underbrace{v_t(Z_t) dt}_{\text{Rectified Flow}} + \underbrace{\sigma_t^2 \nabla \log \rho_t(Z_t) dt + \sqrt{2} \sigma_t dW_t}_{\text{Langevin Dynamics}}, \quad \tilde{Z}_0 = Z_0.$$



$$\nabla \log \rho_t(x) = \frac{\alpha_t v_t(x) - \dot{\alpha}_t x}{\lambda_t \beta_t}, \quad \text{where } \lambda_t = \dot{\alpha}_t \beta_t - \alpha_t \dot{\beta}_t.$$

$$dZ_t = v_t(Z_t) dt + \gamma (\alpha_t v_t(Z_t) - \dot{\alpha}_t Z_t) dt + \sqrt{2 \lambda_t \beta_t \gamma_t} dW_t,$$

where we set $\sigma_t^2 = \lambda_t \beta_t \gamma_t$.

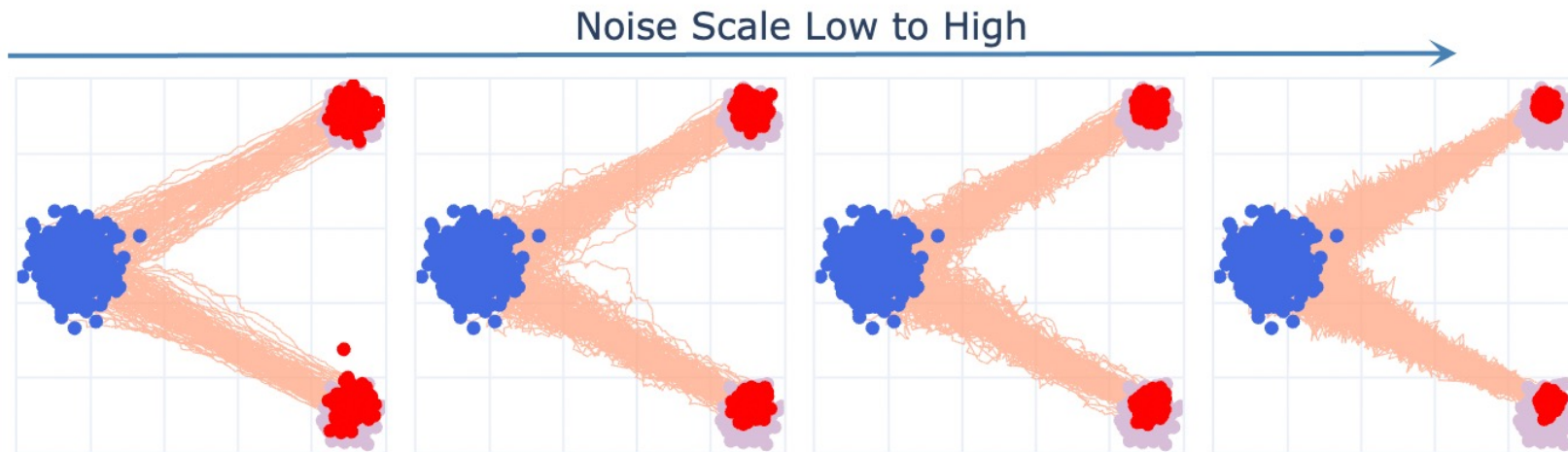
Flow-based models v.s. Diffusion models

- $\nabla \log \rho_t$ itself has **errors in practice**.

$$dZ_t = v_t(Z_t)dt + \gamma(\alpha_t v_t(Z_t) - \dot{\alpha}_t Z_t)dt + \sqrt{2\lambda_t \beta_t \gamma_t} dW_t,$$

where we set $\sigma_t^2 = \lambda_t \beta_t \gamma_t$.

Should not be too large



- Flow-based models
- $\nabla \log \rho_t$ itself has

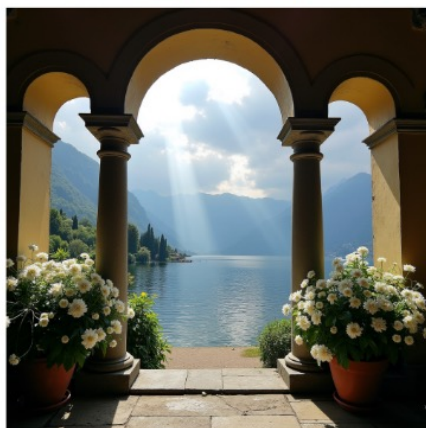
Euler Sampler



Stochastic Sampler



Excessive Langevin



Flow-based models v.s. Diffusion models

Network Output	Formulation	MSE on Network Output
$\hat{\epsilon}$ -prediction	$\hat{\epsilon}$	$\ \hat{\epsilon} - \epsilon\ _2^2$
$\hat{\mathbf{x}}$ -prediction	$\hat{\mathbf{x}} = (\mathbf{z}_t - \sigma_t \hat{\epsilon}) / \alpha_t$	$\ \hat{\mathbf{x}} - \mathbf{x}\ _2^2 = e^{-\lambda} \ \hat{\epsilon} - \epsilon\ _2^2$
$\hat{\mathbf{v}}$ -prediction	$\hat{\mathbf{v}} = \alpha_t \hat{\epsilon} - \sigma_t \hat{\mathbf{x}}$	$\ \hat{\mathbf{v}} - \mathbf{v}\ _2^2 = \alpha_t^2 (e^{-\lambda} + 1)^2 \ \hat{\epsilon} - \epsilon\ _2^2$
$\hat{\mathbf{u}}$ -flow matching vector field	$\hat{\mathbf{u}} = \hat{\epsilon} - \hat{\mathbf{x}}$	$\ \hat{\mathbf{u}} - \mathbf{u}\ _2^2 = (e^{-\lambda/2} + 1)^2 \ \hat{\epsilon} - \epsilon\ _2^2$

Interpolation

- Canonical choice:

$$X_t = tX_1 + (1 - t)X_0.$$

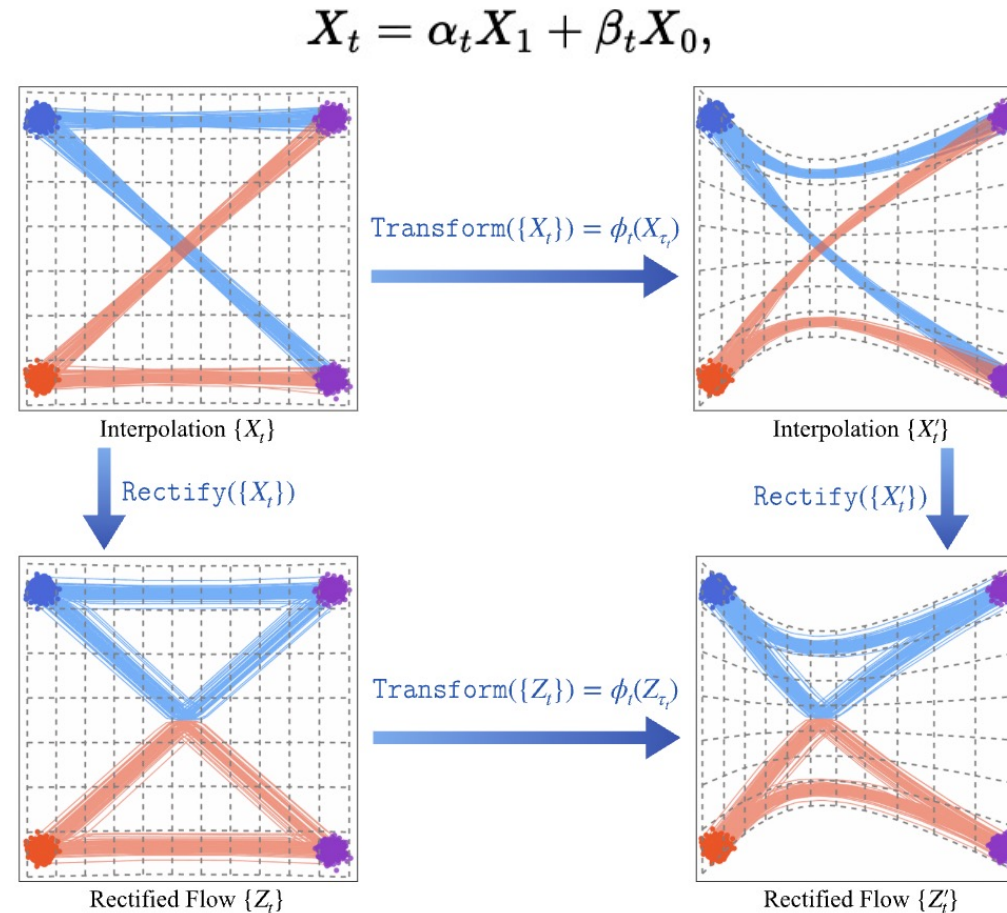
- General form:

$$X_t = \alpha_t X_1 + \beta_t X_0,$$

- Impact of coefficients?

Interpolation

- If two interpolation processes can be deformed into each other with a **differentiable point-wise transform**, trajectories of their rectified flows can also be deformed into each other using **the very same transform**



Interpolation

- If two interpolation processes can be deformed into each other with **a differentiable point-wise transform**, trajectories of their rectified flows can also be deformed into each other using **the very same transform**

$$X_t = \alpha_t X_1 + \beta_t X_0,$$

- All affine interpolations can be pointwisely transformed into one another through **simple time and variable scaling**
 - Can use the simplest straight-line interpolation **during training**, while recovering the rectified flow for all affine interpolations **at inference time**.

Interpolation

- If two interpolation processes can be deformed into each other with **a differentiable point-wise transform**, trajectories of their rectified flows can also be deformed into each other using **the very same transform**

$$X_t = \alpha_t X_1 + \beta_t X_0,$$

- All affine interpolations can be pointwisely transformed into one another through **simple time and variable scaling**
 - Can use the simplest straight-line interpolation **during training**, while recovering the rectified flow for all affine interpolations **at inference time**.
 - Using different affine interpolations during training is equivalent to changing the **time-weighting**

$$\mathcal{L}(\theta) = \int_0^1 \mathbb{E} \left[\eta_t \left\| \dot{X}_t - v_t(X_t; \theta) \right\|^2 \right] dt,$$